

Patrones de Diseño: Factory y Abstract Factory

Estos patrones nos ayudan a desacoplar la creación de objetos del código que los utiliza.

Patrón Factory Method

Permite que una clase delegue la responsabilidad de instanciar objetos a sus subclasses. Es útil cuando tenemos varios tipos de un mismo objeto.

Imaginemos que tenemos ómnibus **Urbano** y de **LargaDistancia**, ambos extienden de **Omnibus**.

PHP

```
// Interfaz para nuestras fábricas
interface OmnibusFactory {
    public function crearOmnibus(int $id, string $linea): Omnibus;
}

// Fábrica concreta para buses urbanos
class FabricaOmnibusUrbano implements OmnibusFactory {
    public function crearOmnibus(int $id, string $linea): Omnibus {
        return new OmnibusUrbano($id, $linea, 'en_deposito'); // Lógica específica
    }
}

// Fábrica concreta para buses de larga distancia
class FabricaOmnibusLargaDistancia implements OmnibusFactory {
    public function crearOmnibus(int $id, string $linea): Omnibus {
        return new OmnibusLargaDistancia($id, $linea, 'listo_para_viaje'); // Lógica específica
    }
}

// Uso
$fabricaUrbana = new FabricaOmnibusUrbano();
$bus123 = $fabricaUrbana->crearOmnibus(123, 'Línea 5');
```

Ventaja: El código cliente solo necesita saber qué fábrica usar, pero no cómo se construye cada tipo de ómnibus.

Patrón Abstract Factory

Es una "fábrica de fábricas". Se utiliza cuando necesitamos crear **familias de objetos relacionados** sin especificar sus clases concretas.

Extendamos el ejemplo: un ómnibus necesita un **Conductor** y un **SistemaDeTickets**. Estos varían si el bus es urbano o de larga distancia.

PHP

// Interfaz de la fábrica abstracta

```
interface FabricaTransporte {  
    public function crearOmnibus(): Omnibus;  
    public function crearConductor(): Conductor;  
}
```

// Implementación para transporte urbano

```
class FabricaTransporteUrbano implements FabricaTransporte {  
    public function crearOmnibus(): Omnibus {  
        return new OmnibusUrbano(...);  
    }  
    public function crearConductor(): Conductor {  
        return new ConductorUrbano(...); // Conductor con licencia local  
    }  
}
```

// Implementación para larga distancia

```
class FabricaTransporteLargaDistancia implements FabricaTransporte {  
    public function crearOmnibus(): Omnibus {  
        return new OmnibusLargaDistancia(...);  
    }  
    public function crearConductor(): Conductor {  
        return new ConductorLargaDistancia(...); // Conductor con licencia interdepartamental  
    }  
}
```

Ventaja: Garantiza que los objetos creados sean compatibles entre sí (un **OmnibusUrbano** siempre se creará junto a un **ConductorUrbano**). El cliente elige una "familia" (la fábrica) y el sistema se encarga de que todo sea coherente.